

trial_1

June 7, 2025

Trial_1 : CNN architecture transformed into a **Multi-Scale Convolutional Neural Network (MSCNN)** using *parallel convolution layers* with different kernel sizes.

```
[2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#from tensorflow.keras.models import Sequential
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, BatchNormalization, Dropout,
    ↳Concatenate, Flatten, Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix,
    ↳roc_auc_score, roc_curve
```

1 load & Split data

```
[3]: # Load feature and target data
features = pd.read_csv(r"D:\JN\data set\created dataset\ECG PCG_
    ↳feature(128+128=256).csv", header=None, skiprows=[0])
targets = pd.read_csv(r'D:\JN\data set\data from web\Reference_aug.csv',
    ↳header=None)

print(features.shape)
print(targets.shape)

# Extract target column
y = targets.iloc[:, -1] # Selecting the last column
X = features # All feature columns

# Using .apply() method
y = y.apply(lambda x: max(x, 0))

# Split data into training and testing sets
Xtrain, Xtest, Ytrain, Ytest = train_test_split(X, y, test_size=0.2,
    ↳random_state=42)
```

```
(1975, 256)
(1975, 2)
```

```
[4]: # Standardize features
scaler = StandardScaler()
Xtrain_scaled = scaler.fit_transform(Xtrain)
Xtest_scaled = scaler.transform(Xtest)

# Reshape data for Conv1D input
Xtrain_reshaped = Xtrain_scaled.reshape((Xtrain_scaled.shape[0], Xtrain_scaled.
↳shape[1], 1))
Xtest_reshaped = Xtest_scaled.reshape((Xtest_scaled.shape[0], Xtest_scaled.
↳shape[1], 1))
```

2 Multi-Scale Convolutional Neural Network (MSCNN)

```
[5]: input_layer = Input(shape=(256, 1))

# Multi-scale convolution block
conv3 = Conv1D(filters=16, kernel_size=3, padding='same',␣
↳activation='relu')(input_layer)
conv5 = Conv1D(filters=16, kernel_size=5, padding='same',␣
↳activation='relu')(input_layer)
conv7 = Conv1D(filters=16, kernel_size=7, padding='same',␣
↳activation='relu')(input_layer)

# Merge multi-scale features
merged = Concatenate(axis=-1)([conv3, conv5, conv7])
merged = BatchNormalization()(merged)
merged = Dropout(0.5)(merged)

# Additional convolution + dropout layers
conv = Conv1D(filters=32, kernel_size=3, activation='relu',␣
↳padding='same')(merged)
conv = BatchNormalization()(conv)
conv = Dropout(0.5)(conv)

conv = Conv1D(filters=16, kernel_size=3, activation='relu',␣
↳padding='same')(conv)

# Fully connected layers
flattened = Flatten()(conv)
dense = Dense(128, activation='relu')(flattened)
dense = Dense(128, activation='relu')(dense)
dense = Dense(128, activation='relu')(dense)
dense = Dense(128, activation='relu')(dense)
```

```

output = Dense(1, activation='sigmoid')(dense) # For binary classification

# Create and compile model
model = Model(inputs=input_layer, outputs=output)
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

```

```

[6]: # Train model
history = model.fit(Xtrain_resaped, Ytrain, epochs=60, batch_size=32,
validation_split=0.2)

```

```

Epoch 1/60
40/40      8s 78ms/step -
accuracy: 0.5393 - loss: 0.7055 - val_accuracy: 0.6108 - val_loss: 0.6764
Epoch 2/60
40/40      2s 53ms/step -
accuracy: 0.6238 - loss: 0.6510 - val_accuracy: 0.6487 - val_loss: 0.6659
Epoch 3/60
40/40      2s 42ms/step -
accuracy: 0.6495 - loss: 0.6151 - val_accuracy: 0.6487 - val_loss: 0.6555
Epoch 4/60
40/40      2s 48ms/step -
accuracy: 0.6932 - loss: 0.5861 - val_accuracy: 0.6044 - val_loss: 0.6505
Epoch 5/60
40/40      3s 44ms/step -
accuracy: 0.7149 - loss: 0.5479 - val_accuracy: 0.6994 - val_loss: 0.6086
Epoch 6/60
40/40      2s 42ms/step -
accuracy: 0.7547 - loss: 0.5006 - val_accuracy: 0.6962 - val_loss: 0.6003
Epoch 7/60
40/40      2s 46ms/step -
accuracy: 0.7512 - loss: 0.4976 - val_accuracy: 0.6266 - val_loss: 0.7062
Epoch 8/60
40/40      2s 45ms/step -
accuracy: 0.7582 - loss: 0.5118 - val_accuracy: 0.6804 - val_loss: 0.6123
Epoch 9/60
40/40      2s 40ms/step -
accuracy: 0.7975 - loss: 0.4410 - val_accuracy: 0.6867 - val_loss: 0.5959
Epoch 10/60
40/40      2s 42ms/step -
accuracy: 0.7903 - loss: 0.4248 - val_accuracy: 0.7089 - val_loss: 0.5818
Epoch 11/60
40/40      2s 41ms/step -
accuracy: 0.8422 - loss: 0.3803 - val_accuracy: 0.7247 - val_loss: 0.5747
Epoch 12/60
40/40      2s 42ms/step -
accuracy: 0.8403 - loss: 0.3517 - val_accuracy: 0.7120 - val_loss: 0.6094
Epoch 13/60

```

40/40 2s 48ms/step -
 accuracy: 0.8663 - loss: 0.3099 - val_accuracy: 0.7310 - val_loss: 0.6326
 Epoch 14/60
 40/40 2s 43ms/step -
 accuracy: 0.8751 - loss: 0.3012 - val_accuracy: 0.7310 - val_loss: 0.7515
 Epoch 15/60
 40/40 2s 47ms/step -
 accuracy: 0.9016 - loss: 0.2259 - val_accuracy: 0.7342 - val_loss: 0.7655
 Epoch 16/60
 40/40 2s 44ms/step -
 accuracy: 0.8959 - loss: 0.2215 - val_accuracy: 0.7247 - val_loss: 0.9276
 Epoch 17/60
 40/40 2s 48ms/step -
 accuracy: 0.9132 - loss: 0.1975 - val_accuracy: 0.7278 - val_loss: 0.8867
 Epoch 18/60
 40/40 2s 49ms/step -
 accuracy: 0.9292 - loss: 0.1838 - val_accuracy: 0.7405 - val_loss: 0.9490
 Epoch 19/60
 40/40 2s 44ms/step -
 accuracy: 0.9501 - loss: 0.1483 - val_accuracy: 0.7089 - val_loss: 1.1154
 Epoch 20/60
 40/40 2s 47ms/step -
 accuracy: 0.9394 - loss: 0.1404 - val_accuracy: 0.7120 - val_loss: 1.0989
 Epoch 21/60
 40/40 2s 37ms/step -
 accuracy: 0.9408 - loss: 0.1711 - val_accuracy: 0.7089 - val_loss: 1.0859
 Epoch 22/60
 40/40 2s 44ms/step -
 accuracy: 0.9667 - loss: 0.0974 - val_accuracy: 0.7247 - val_loss: 1.0015
 Epoch 23/60
 40/40 2s 38ms/step -
 accuracy: 0.9729 - loss: 0.0710 - val_accuracy: 0.7532 - val_loss: 1.1759
 Epoch 24/60
 40/40 2s 38ms/step -
 accuracy: 0.9740 - loss: 0.0626 - val_accuracy: 0.7310 - val_loss: 1.1090
 Epoch 25/60
 40/40 2s 39ms/step -
 accuracy: 0.9690 - loss: 0.1031 - val_accuracy: 0.7437 - val_loss: 1.0872
 Epoch 26/60
 40/40 2s 40ms/step -
 accuracy: 0.9860 - loss: 0.0425 - val_accuracy: 0.7405 - val_loss: 1.2770
 Epoch 27/60
 40/40 2s 39ms/step -
 accuracy: 0.9728 - loss: 0.0865 - val_accuracy: 0.7342 - val_loss: 1.2830
 Epoch 28/60
 40/40 2s 42ms/step -
 accuracy: 0.9729 - loss: 0.0761 - val_accuracy: 0.7152 - val_loss: 1.2260
 Epoch 29/60

40/40 2s 48ms/step -
 accuracy: 0.9723 - loss: 0.0786 - val_accuracy: 0.7405 - val_loss: 0.9550
 Epoch 30/60
 40/40 2s 45ms/step -
 accuracy: 0.9831 - loss: 0.0492 - val_accuracy: 0.7215 - val_loss: 1.3274
 Epoch 31/60
 40/40 2s 47ms/step -
 accuracy: 0.9865 - loss: 0.0437 - val_accuracy: 0.7500 - val_loss: 1.2100
 Epoch 32/60
 40/40 2s 49ms/step -
 accuracy: 0.9690 - loss: 0.0819 - val_accuracy: 0.7120 - val_loss: 1.1295
 Epoch 33/60
 40/40 2s 50ms/step -
 accuracy: 0.9854 - loss: 0.0421 - val_accuracy: 0.6994 - val_loss: 1.3167
 Epoch 34/60
 40/40 2s 41ms/step -
 accuracy: 0.9869 - loss: 0.0400 - val_accuracy: 0.7278 - val_loss: 1.4437
 Epoch 35/60
 40/40 3s 51ms/step -
 accuracy: 0.9771 - loss: 0.0656 - val_accuracy: 0.7373 - val_loss: 1.3524
 Epoch 36/60
 40/40 3s 47ms/step -
 accuracy: 0.9904 - loss: 0.0257 - val_accuracy: 0.7247 - val_loss: 1.4817
 Epoch 37/60
 40/40 2s 51ms/step -
 accuracy: 0.9863 - loss: 0.0496 - val_accuracy: 0.7089 - val_loss: 1.4544
 Epoch 38/60
 40/40 2s 53ms/step -
 accuracy: 0.9942 - loss: 0.0182 - val_accuracy: 0.7215 - val_loss: 1.5985
 Epoch 39/60
 40/40 2s 53ms/step -
 accuracy: 0.9950 - loss: 0.0165 - val_accuracy: 0.6835 - val_loss: 1.6959
 Epoch 40/60
 40/40 2s 43ms/step -
 accuracy: 0.9905 - loss: 0.0267 - val_accuracy: 0.7342 - val_loss: 1.5783
 Epoch 41/60
 40/40 2s 56ms/step -
 accuracy: 0.9909 - loss: 0.0385 - val_accuracy: 0.7057 - val_loss: 1.5599
 Epoch 42/60
 40/40 2s 53ms/step -
 accuracy: 0.9868 - loss: 0.0439 - val_accuracy: 0.7215 - val_loss: 1.5325
 Epoch 43/60
 40/40 2s 40ms/step -
 accuracy: 0.9900 - loss: 0.0339 - val_accuracy: 0.7184 - val_loss: 1.5037
 Epoch 44/60
 40/40 2s 44ms/step -
 accuracy: 0.9898 - loss: 0.0348 - val_accuracy: 0.7247 - val_loss: 1.5635
 Epoch 45/60

40/40 2s 46ms/step -
 accuracy: 0.9913 - loss: 0.0184 - val_accuracy: 0.7373 - val_loss: 1.6257
 Epoch 46/60
 40/40 2s 46ms/step -
 accuracy: 0.9912 - loss: 0.0228 - val_accuracy: 0.7247 - val_loss: 1.3251
 Epoch 47/60
 40/40 2s 46ms/step -
 accuracy: 0.9957 - loss: 0.0141 - val_accuracy: 0.7373 - val_loss: 1.4160
 Epoch 48/60
 40/40 2s 52ms/step -
 accuracy: 0.9963 - loss: 0.0107 - val_accuracy: 0.7310 - val_loss: 1.6704
 Epoch 49/60
 40/40 2s 42ms/step -
 accuracy: 0.9901 - loss: 0.0244 - val_accuracy: 0.7152 - val_loss: 1.6113
 Epoch 50/60
 40/40 2s 47ms/step -
 accuracy: 0.9790 - loss: 0.0613 - val_accuracy: 0.6962 - val_loss: 1.2264
 Epoch 51/60
 40/40 2s 47ms/step -
 accuracy: 0.9891 - loss: 0.0385 - val_accuracy: 0.7437 - val_loss: 1.3656
 Epoch 52/60
 40/40 2s 43ms/step -
 accuracy: 0.9950 - loss: 0.0151 - val_accuracy: 0.7342 - val_loss: 1.6185
 Epoch 53/60
 40/40 2s 41ms/step -
 accuracy: 0.9933 - loss: 0.0143 - val_accuracy: 0.7532 - val_loss: 1.8967
 Epoch 54/60
 40/40 2s 42ms/step -
 accuracy: 0.9970 - loss: 0.0093 - val_accuracy: 0.7310 - val_loss: 1.8596
 Epoch 55/60
 40/40 2s 55ms/step -
 accuracy: 0.9895 - loss: 0.0288 - val_accuracy: 0.7247 - val_loss: 1.6956
 Epoch 56/60
 40/40 2s 39ms/step -
 accuracy: 0.9906 - loss: 0.0232 - val_accuracy: 0.7342 - val_loss: 1.5671
 Epoch 57/60
 40/40 2s 43ms/step -
 accuracy: 0.9913 - loss: 0.0221 - val_accuracy: 0.7247 - val_loss: 1.7938
 Epoch 58/60
 40/40 2s 49ms/step -
 accuracy: 0.9943 - loss: 0.0118 - val_accuracy: 0.7342 - val_loss: 1.8999
 Epoch 59/60
 40/40 2s 43ms/step -
 accuracy: 0.9955 - loss: 0.0166 - val_accuracy: 0.6962 - val_loss: 1.8215
 Epoch 60/60
 40/40 2s 48ms/step -
 accuracy: 0.9971 - loss: 0.0125 - val_accuracy: 0.7184 - val_loss: 1.7452

```
[7]: # Evaluate the model on the test set
loss, accuracy = model.evaluate(Xtest_resaped, Ytest)
print(f"Test Accuracy: {accuracy:.2%}")
print(f"Test Loss: {loss:.4f}")
```

```
13/13          0s 12ms/step -
accuracy: 0.6937 - loss: 2.1317
Test Accuracy: 70.38%
Test Loss: 2.0569
```

```
[8]: Ypred = model.predict(Xtest_resaped)
Ypred_classes = (Ypred > 0.5).astype(int)
Ypred_classes_flat = Ypred_classes.flatten()

# Print classification report (precision, recall, F1-score)
print("\nClassification Report:")
print(classification_report(Ytest, Ypred_classes_flat)) # Use Ytest directly
```

```
13/13          1s 28ms/step
```

Classification Report:

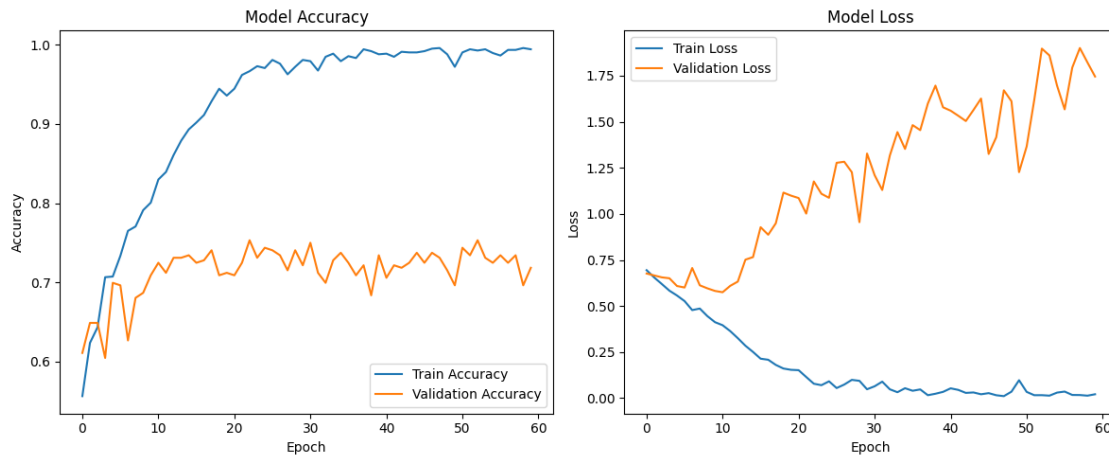
	precision	recall	f1-score	support
0	0.67	0.76	0.71	190
1	0.75	0.65	0.69	205
accuracy			0.70	395
macro avg	0.71	0.71	0.70	395
weighted avg	0.71	0.70	0.70	395

```
[9]: # Plot accuracy
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.legend()

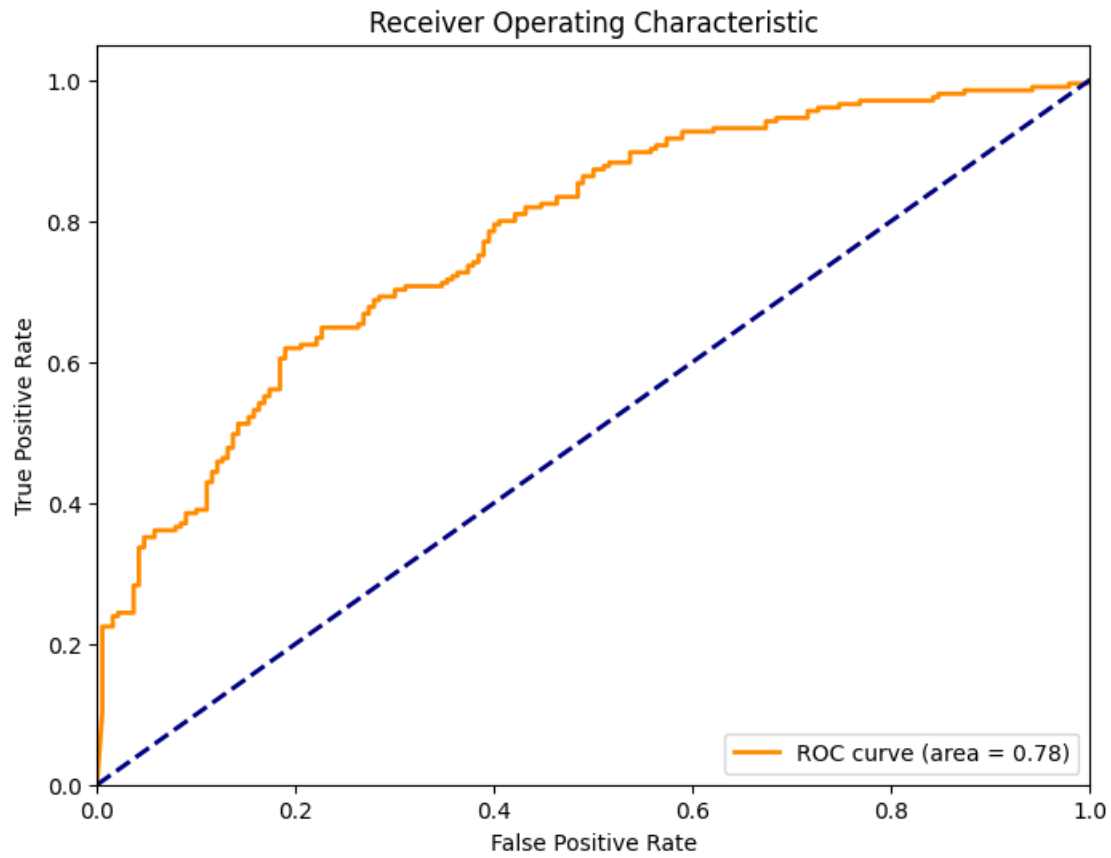
plt.tight_layout()
plt.show()
```



```
[10]: roc_auc = roc_auc_score(Ytest, Ypred[:, 0])
print(f"\nROC AUC: {roc_auc:.4f}")

# Plot ROC curve
# Use Ytest for true labels and Ypred[:, 0] for predicted probabilities of the
# positive class
fpr, tpr, thresholds = roc_curve(Ytest, Ypred[:, 0])
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (area = %0.2f)' %
# roc_auc)
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```

ROC AUC: 0.7776



```
[11]: cm = confusion_matrix(Ytest, Ypred_classes_flat)
plt.figure(figsize=(8, 6))
class_labels = ['Class 0', 'Class 1'] # Or provide your actual class names if
↳available

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

